

# Dive into Deep Learning (D2L)

## Recurrent Neural Networks

---

Joon Soo Yoo, Ph.D.

Spring 2026

Assistant Professor

Defense & Advanced Security (DAS) Lab

Jeonbuk National University (JBNU)

# Recurrent Neural Networks

# 1. From Fixed-Length Data to Sequential Data

- 지금까지 다룬 대부분의 모델:
  - Linear Regression
  - Logistic Regression
  - Multilayer Perceptron (MLP)
  - Convolutional Neural Network (CNN)
- 공통점:
  - 입력 크기(input dimension)가 고정됨
- 예:
  - Fashion-MNIST: 항상  $28 \times 28$
  - 이미지 하나 입력 → 하나의 예측 출력

## Question

그렇다면 sequence 데이터는 어떻게 처리할까?

## 2. Sequential Data is Everywhere

- 현실 세계의 많은 데이터는 sequence 구조를 가짐

| Application         | Sequential Structure  |
|---------------------|-----------------------|
| Text                | Word sequence         |
| Speech              | Audio sequence        |
| Video               | Frame sequence        |
| Music               | Temporal signal       |
| Satellite Telemetry | Sensor sequence       |
| Robot Control       | State-action sequence |

### Important

Sequence 데이터에서는 순서(order)와 문맥(context)이 중요함

### 3. What Makes Sequential Data Difficult?

- 기존 tabular 데이터에서는:
  - 각 feature가 독립적이라고 가정 가능
- 하지만 sequence 데이터에서는:
  - 현재 입력이 과거 입력에 의존함

#### Example

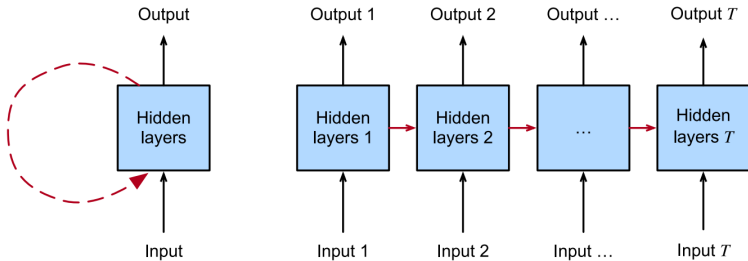
"I love this movie" 와  
"This movie love I" 는  
같은 단어를 사용하지만 의미가 완전히 다름

#### Core Challenge

이전 정보를 기억하면서 현재 데이터를 처리해야 함

## 4. Recurrent Neural Networks (RNNs)

- RNN은 sequence dynamics를 모델링하기 위한 neural network
- 핵심 아이디어:
  - 이전 상태(hidden state)를 다음 timestep으로 전달



왼쪽: cyclic recurrent structure

오른쪽: time-unrolled representation

## 5. Unrolling Through Time

- RNN은 시간축(time axis) 방향으로 펼쳐서 이해 가능
- 각 timestep에서:
  - 동일한 parameter(shared parameter)를 사용

$$x_1 \rightarrow h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow \dots$$

### Key Property

모든 timestep에서 같은 neural network를 반복 사용함

## 6. Sequence Representation

- Sequence 입력:

$$x_1, x_2, x_3, \dots, x_T$$

- 각 입력은 timestep index를 가짐
- sequence 길이  $T$  는 고정되지 않을 수 있음

### Examples

- 문장: 단어들의 sequence
- 환자 기록: 이벤트 sequence
- 위성 telemetry: 센서 sequence



## 7. Sequence Dependency

- Sequence에서는 timestep 간 dependency가 존재

$$P(x_t \mid x_{t-1}, x_{t-2}, \dots, x_1)$$

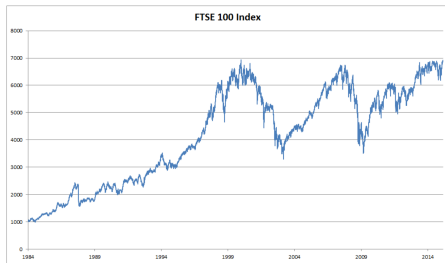
- 현재 값은 과거 정보(history)에 의존
- 이를 sequence modeling이라고 함

### Example

다음 단어 예측: "I love deep" → "learning"

## 8. Autoregressive Modeling

- 과거 값을 이용하여 미래 값을 예측



FTSE 100 stock price sequence

- 목표:

$$P(x_t \mid x_{t-1}, \dots, x_1)$$

- 이러한 접근을 autoregressive modeling이라고 부름

## 9. The Problem of Variable-Length Inputs

- 시간이 증가할수록 입력 길이도 증가

$x_1$

$x_1, x_2$

$x_1, x_2, x_3$

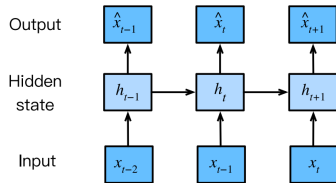
...

### Challenge

일반적인 MLP는 fixed-size input을 요구함  
→ sequence 전체를 직접 처리하기 어려움

# 10. Hidden State Representation

- 해결 아이디어:
  - 과거 sequence를 hidden state에 압축 저장



$$h_t = g(h_{t-1}, x_{t-1})$$

$$\hat{x}_t = P(x_t | h_t)$$

## 11. Interpretation of Hidden State

- Hidden state:
  - 과거 정보를 요약한 latent representation
- 새로운 입력이 들어오면:
  - hidden state 업데이트
  - 다음 prediction 생성

### Interpretation

Hidden state는 “지금까지 읽은 sequence의 압축된 memory” 라고 볼 수 있음

## 12. Summary

- Sequence 데이터는 시간 의존성(time dependency)을 가짐
- 기존 MLP/CNN은 sequence 처리에 한계 존재
- RNN은 hidden state를 이용하여 과거 정보를 유지함
- 핵심 목표:
  - 과거 정보를 기반으로 미래를 예측

### Next Step

다음에는 실제 RNN 구조와 language model을 살펴봄

# Recurrent Neural Networks (RNNs)

Part 2

# 1. Limitation of n-gram Models

- 기존 n-gram language model:

$$P(x_t \mid x_{t-1}, \dots, x_{t-n+1})$$

- 문제:
  - 더 긴 문맥(context)을 보려면  $n$  증가 필요

## Parameter Explosion

Vocabulary size가  $|V|$  일 때,  
필요한 경우의 수:

$$|V|^n$$

예:

$$10000^3$$

→ 매우 큰 메모리와 데이터 필요



## 2. Core Idea: Hidden State

- 모든 과거 정보를 직접 저장하지 말고
- compressed memory(hidden state)를 사용

$$P(x_t \mid x_{t-1}, \dots, x_1) \approx P(x_t \mid h_{t-1})$$

### Interpretation

$h_{t-1}$  는

현재까지의 sequence 정보를 압축한 latent representation

### 3. Hidden State Update

- Hidden state는:
    - 현재 입력
    - 이전 hidden state
- 를 이용하여 업데이트됨

$$h_t = f(x_t, h_{t-1})$$

#### Meaning

- $x_t$ : 현재 입력
- $h_{t-1}$ : 이전 memory
- $h_t$ : 업데이트된 새로운 memory

## 4. Hidden Layer vs Hidden State

- Hidden Layer 와 Hidden State 는 서로 다른 개념

| Concept      | Meaning               |
|--------------|-----------------------|
| Hidden Layer | 입력과 출력 사이의 layer      |
| Hidden State | sequence memory/state |

### Important

Hidden state는 이전 timestep의 정보를 유지함

## 5. MLP without Hidden State

- 기존 MLP는 memory가 없음
- 현재 입력만 사용

$$H = \phi(XW_{xh} + b_h)$$

$$O = HW_{hq} + b_q$$

### Problem

과거 timestep 정보를 자동으로 기억하지 못함

## 6. RNN with Hidden State

- RNN은 이전 hidden state를 현재 계산에 사용

$$H_t = \phi(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$$

### Key Difference

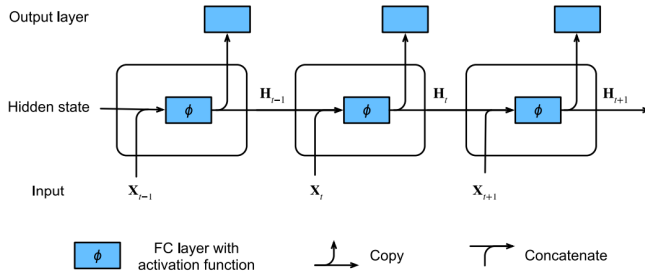
새로운 항:

$$H_{t-1} W_{hh}$$

→ 과거 정보(memory)를 현재 timestep으로 전달

## 7. Recurrent Computation

- Hidden state는 timestep마다 재귀적으로(recurrently) 업데이트됨



RNN hidden state computation across adjacent timesteps

## 8. Why RNN Has Memory

- Hidden state는 이전 timestep의 정보를 포함

$$H_1 \rightarrow H_2 \rightarrow H_3 \rightarrow \dots$$

### Interpretation

$H_t$  는 현재까지 sequence의 compressed summary 역할

### Contrast

MLP: 현재만 봄

RNN: 현재 + 과거 기억 같이 사용

## 9. Parameter Sharing

- 모든 timestep에서 동일한 parameter 사용

$$W_{xh}, W_{hh}, W_{hq}$$

- timestep이 증가해도 parameter 수는 증가하지 않음

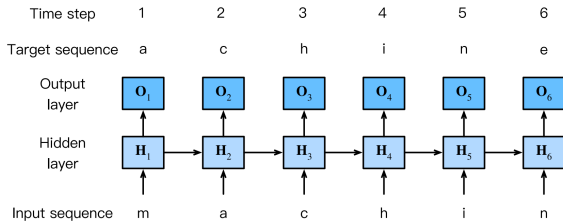
### Benefit

긴 sequence도 같은 모델 구조로 처리 가능



## 10. Character-Level Language Model

- RNN은 다음 character prediction에 사용 가능



Input: "machin"

Target: "achine"

### Goal

현재까지의 character sequence를 이용하여 다음 character 예측

## 11. Strength and Limitation of RNN

- 장점:
  - sequence dependency 학습 가능
  - parameter sharing 가능
  - variable-length sequence 처리 가능
- 한계:
  - hidden state 하나에 정보를 압축
  - 긴 sequence에서 정보 손실 발생 가능

### Result

Long-term dependency 학습이 어려움

## 12. Motivation for LSTM

- RNN은 오래된 정보를 잘 기억하지 못함
- gradient vanishing 문제가 발생 가능

### Question

어떻게 중요한 정보는 오래 유지하고,  
불필요한 정보는 제거할 수 있을까?

### Next Topic

LSTM: gated memory architecture

# Long Short-Term Memory (LSTM)

# 1. Motivation for LSTM

- Vanilla RNN의 주요 문제:
  - Vanishing Gradient
  - Exploding Gradient
  - Long-term dependency 학습 어려움
- 긴 sequence에서는 오래된 정보가 점점 사라짐

## Core Question

중요한 정보는 오래 기억하고,  
불필요한 정보는 잊을 수 있을까?

## 2. Core Idea of LSTM

- LSTM은 RNN에 memory cell을 추가
- 핵심:
  - information flow를 gate로 제어

| Component | Role                           |
|-----------|--------------------------------|
| $H_t$     | hidden state (output)          |
| $C_t$     | memory cell (long-term memory) |

### Interpretation

- Hidden state: 현재 timestep output
- Cell state: 장기 memory 저장소

### 3. Gated Memory Cell

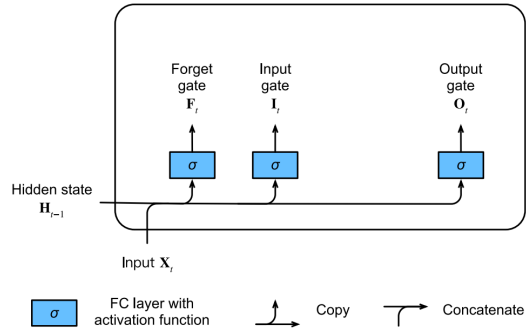
- LSTM은 여러 개의 gate를 사용

| Gate        | Role            |
|-------------|-----------------|
| Input Gate  | 새로운 정보 저장 여부    |
| Forget Gate | 기존 memory 유지 여부 |
| Output Gate | memory 출력 여부    |

#### Key Property

Gate 값은 sigmoid를 통해 0 ~ 1 범위를 가짐

## 4. Input, Forget, and Output Gates



Three gates in an LSTM: input gate, forget gate, and output gate



## 5. Forget Gate

- Forget gate: 이전 memory를 얼마나 유지할지 결정

$$F_t = \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f)$$

### Interpretation

- $F_t \approx 1$ : memory 유지
- $F_t \approx 0$ : memory 삭제

## 6. Input Gate

- Input gate: 새로운 정보를 memory에 얼마나 반영할지 결정

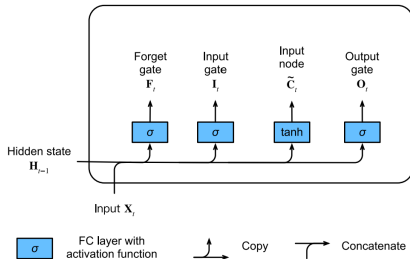
$$I_t = \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i)$$

### Meaning

새로운 정보의 중요도를 학습

## 7. Candidate Memory

- 새롭게 저장할 memory 후보 생성



$$\tilde{C}_t = \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c)$$

## 8. Memory Cell Update

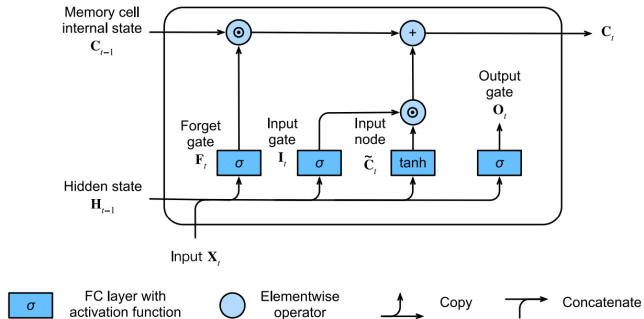
- 이전 memory와 새로운 정보를 결합

$$C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t$$

### Interpretation

- 첫 번째 항: 기존 memory 유지
- 두 번째 항: 새로운 정보 추가

## 9. Memory Flow in LSTM



Updating the internal memory cell state

## 10. Why LSTM Helps

- 만약:

$$F_t \approx 1$$

이면:

$$C_t \approx C_{t-1}$$

- 즉: memory가 거의 그대로 전달됨

### Important

Gradient가 장기간 유지될 수 있어 vanishing gradient 문제가 완화됨

## 11. Output Gate and Hidden State

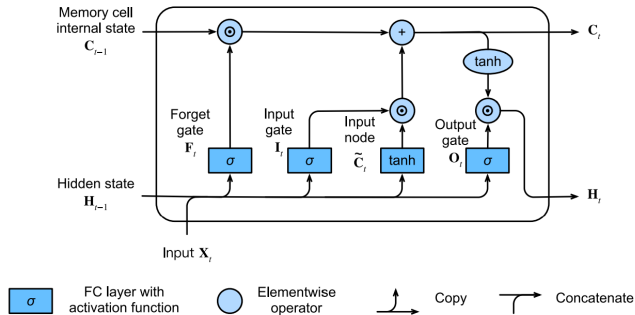
- Output gate: memory를 외부에 얼마나 출력할지 결정

$$H_t = O_t \odot \tanh(C_t)$$

### Interpretation

- memory는 내부에 유지 가능
- 필요할 때만 hidden state로 출력

## 12. Data Flow in LSTM



Computing the hidden state in an LSTM model



## 13. Intuition Behind LSTM

- Vanilla RNN:
  - 하나의 hidden state만 사용
  - 긴 sequence에서 memory 손실 가능
- LSTM:
  - memory cell 유지
  - gate를 통해 정보 흐름 제어

### Result

긴 문맥(long-term dependency)을 더 안정적으로 학습 가능

## 14. Summary

- LSTM은 RNN의 memory 문제를 해결하기 위해 등장
- 핵심:
  - Forget Gate
  - Input Gate
  - Output Gate
- Gate를 통해:
  - 중요한 정보 유지
  - 불필요한 정보 제거

Any Question?